

Snakemake: An Overview

Emil Cacayan

This tutorial was inspired by this YouTube Video:

- [Introducing Snakemake \(Edinburgh Genomics Training\)](#)
-

Snakemake

“The Snakemake workflow management system is a tool to create **reproducible and scalable** data analyses. Workflows are described via a human readable, Python based language.

They can be seamlessly scaled to server, cluster, grid, and cloud environments, without the need to modify the workflow definition. Snakemake workflows can entail a description of required software, which will be automatically deployed to any execution environment.

Finally, workflow runs can be automatically turned into interactive portable browser based reports, which can be shared with collaborators via email or the cloud and combine results with all used parameters, code, and software.”

Source: [Official Snakemake Documentation](#)

Snakemake is powerful than simply typing script managers for bioinformatics workflows in two ways:

1. Workflow definitions can be imported into new environments (increased code portability)
 2. In bioinformatics, many tools are often used - you can use Snakemake to define tools and deploy/test as needed integrating with the conda package management system
-

History of Snakemake

Based on **Unix make**, which is used to build software from source code (by Stuart Feldman in 1976 at Bell Labs, became standard build tool across Unix and Unix-like machines). Bioinformaticians realized that this could be used to write workflows, but they ran into several limitations, one of which is that the syntax is incredibly unintuitive.

Johannes Köster at University Hospital Essen while as a PhD student wrote a new piece of software based on make implemented in Python (hence Snakemake) and is still widely used today.

The version 1.0 release was in April 2012 and is constantly being updated (the current most stable version as of the writing of this document is 9.21.1, released on May 29, 2026). This is an entirely free software and is available on Mac, Linux, and Windows.

Advantages of Snakemake

Using a workflow manager like Snakemake provides significant improvements to scalability, error recovery, and transparency over writing linear scripts (like Shell, Bash, or Python). While a basic master script runs commands sequentially from top to bottom, Snakemake operates on explicitly declared, file-driven logic. Essentially, you define desired outputs and rules to make them, and Snakemake builds a Directed Acyclic Graph (DAG) to handle the rest. More specifically,

1. Smart Re-Entry and Incremental Builds

In a standard script, you must comment out finished steps manually or write out messy conditional `if file_exists:` checks around every command - Snakemake does this automatically. Snakemake tracks file modification timestamps,

checking if outputs are outdated and if a rule needs to be re-run. In addition, if the pipeline fails at a midpoint in the workflow, snakemake is able to continue the workflow at the same point following revision of the error and restarting allowing for much more efficient test-driven-development. If you change an early dataset, Snakemake only reruns downstream steps affected by that specific change.

2. “Free” and Automatic Parallelization

Snakemake analyzes the workflow’s dependency graph to run independent steps simultaneously, and each rule can be specified core limits. If you tell Snakemake to use 16 cores, it will dynamically juggle tasks to keep all 16 cores busy without exceeding the limit. **Note from author:** Coming from a background in object-oriented code, this is a fairly non-trivial task. Writing this type of code requires complex, error-prone multithreading or subprocess pooling.

3. Separation of Pipeline Logic from Infrastructure

You can run the exact same Snakefile on a laptop, an HPC cluster, or a cloud compute instance. It natively submits jobs to schedulers such as Slurm or PBS and is able to handle job dependencies automatically. Moving a standard script from a laptop to a cluster usually involves completely refactoring code to use different types of job arrays, changing file paths, and hardcoding cluster submission commands. Snakemake can do all these things automagically.

4. Built-in Environment and Software Management

Do you hate having to revolve around .yaml files for your conda environments? So do I! Snakemake can link conda environments, Docker containers, or Apptainer/Singularity images directly to individual rules. That way, when a different machine runs the pipeline, Snakemake automatically downloads and installs correct software versions for each step, enhancing reproducibility significantly.

5. Scaling via Wildcards

Instead of writing a loop over 100 samples, you can use wildcards. Snakemake automatically expands wildcards based on input files, applying the rule to all matching datasets simultaneously - standard scripts rely on nested forloops that easily break if file names vary or if samples are added or removed.

6. Safety, Visibility, and Debugging

Running `snakemake -n` performs a dry run, mapping out the entire plan and catching syntax or file path errors before spending time or money on computing. If a step crashes midway through execution, Snakemake automatically deletes steps from accidentally using bad data. It can automatically generate a visual diagram of your pipeline’s workflow or export detailed execution statistics and execution benchmarks.

How Does it Work?

Snakemake essentially works as a script - but doesn’t run as a script in a traditional sense. Essentially you write a modified API for your pipeline in a type of file called a **Snakefile**. In this file, you specify a set of **rules**, all of which have explicitly defined **inputs**, **outputs**, and **actions**. Snakemake is able to work out the correct order of rules to reach a given **target**, then executes them.

Don’t worry, the explanation for what that means is forthcoming.

How to Run Snakemake

Let’s say we have some directory, called `example_directory`. We can navigate into this directory,

```
$cd example_directory
```

And if we inspect the hypothetical contents of this file:

```
$ls
data1.csv data2.csv plotter Snakefile
```

The contents of the file are as follows:

- **data1.csv:** Some dataset.
- **data2.csv:** Also some dataset.
- **plotter:** A script which is designed to plot the .csv data.

If we then run snakemake, we get the following output:

```
$snakemake data_1plot.pdf
Building DAG of jobs...
Using shell: /bin/bash
Provided cores: 1
Rules claiming more threads will be scaled down.
Job counts:
   count  jobs
     1     plot
     1

[Tues Jun  2 12:18:25 2026]
rule plot:
  input: data1.csv
  output: data1_plot.pdf
  jobid: 0
  wildcards: dataset=data1

[Tues Jun  2 12:18:26 2026]
Finished job 0.
1 of 1 steps (100%) done
```

So essentially, Snakemake knew that the data had to be processed by the script and then outputted as a .pdf. How did Snakemake know how to do this?

Here's what that Snakefile looks like:

```
rule plot:
  output: "{dataset}_plot.pdf"
  input:  "{dataset}.csv"
  shell:  " ./plotter -o {output} {input}"

rule filter:
  output: "{csvdata}_filtered.csv"
  input:  "{csvdata}.csv"
  "egrep -v ^boring {input} > {output}"
```

This file spans 8 lines and contains two rules. The syntax and semantics of a Snakemake file is very similar to Python (for those familiar). We see two rules, `plot` and `filter`, each containing an output, an input, and `shell` (which is the command which is to be run). Snakemake looks at the file it is trying to make, looks at the rule in the Snakefile, and links rules together where the outputs and inputs overlaps, and enters the inputs into the shell command templates.

So let's say that we wanted to generate `data1_plot.pdf`. Intuitively, we see that the only rule we need is `plot`. The `{dataset}` parameter becomes `data1`, thus the input is `data1.csv`, the output is `data1_plot.pdf`, and the shell command is `./plotter -o data1_plot.pdf data1.csv`. Snakemake automatically figures out that this is the correct tool to use.

But what about that second rule? This is where the power of Snakemake really comes into full view.

Let's say now that we wanted to generate a file called `data1_filtered_plot.pdf`. To get there, the final step has to be an output from the `plot` rule, our input then being `data1_filtered.csv`. But from our `ls` command:

```
$ls
data1.csv data2.csv plotter Snakefile
```

no such file exists. So now Snakemake looks for a way to output a file, and sees that rule `filter` outputs a file with a suffix `_filtered.csv`. The input would then be `data1.csv`, and the output `data1_filtered.csv`. Once the shell command from `filter` is run, the `data1_filtered.csv` command is generated and we can successfully run the `plot` rule.

Normally, we'd have to hard code all these connections and naming conventions. Snakemake is able to use the Snakefile and create a directed acyclic graph before executing the pipeline - i.e. it is able to pattern match the pipeline structure without us having to do any work!

Useful Options

- `-p`: Prints shell commands.
- `-n`: Only show the steps of the final pipeline, don't run.
- `-F`: Force run all steps, skips file timestamp checks.
- `-j` or `--cores`: Run multiple jobs in parallel. Running `snakemake --cores 4` tells Snakemake to run up to 4 parallel local jobs. In cluster or cloud environments, using `--jobs 100` tells Snakemake it can submit up to 100 jobs simultaneously to the scheduler.

N.B.

This is an incredibly simple walkthrough of Snakemake - for a more robust and thorough explanation of all Snakemake features, view the official documentation [here](#).

References

Mölder, F., Jablonski, K. P., Letcher, B., Hall, M. B., van Dyken, P. C., Tomkins-Tinch, C. H., Sochat, V., Forster, J., Vieira, F. G., Meesters, C., Lee, S., Twardziok, S. O., Kanitz, A., VanCampen, J., Malladi, V., Wilm, A., Holtgrewe, M., Rahmann, S., Nahnsen, S., & Köster, J. (2021). Sustainable data analysis with Snakemake. *F1000Research*, 10, 33. <https://doi.org/10.12688/f1000research.29032.3>